

Christian_Herrera_HW7

March 10, 2026

1 Homework 7: OOP, File I/O, Exception (Questions)

2 Assignment Submission Guidelines

Please follow the guidelines below for submitting your assignment:

1. Submission Deadline:

- All assignments must be submitted **no later than 23:59 PM next Tuesday (03/10)**.
- Late submissions will not be accepted unless prior arrangements have been made by the TAs.

2. Complete Your Work:

- Finish your work in Google Colab and ensure that your notebook is saved.

3. Submit Your Assignment on Canvas:

- Log in to **Canvas** and navigate to the assignment submission page.

4. File Naming Convention:

- Please name your files as follows: `Lastname_Firstname_AssignmentName`
- Example: `Alex_John_HW7.ipynb` and `Alex_John_HW7.pdf`

5. Technical Issues:

- If you encounter any technical issues with Canvas or your submission, please contact the TAs immediately **before the deadline** to avoid penalties.

3 Questions

3.1 Question 1. The Electronics Product Hierarchy (30 Points)

3.1.1 Tasks:

1. Product Class (Superclass):

- Define a superclass named **Product** with the following attributes:
 - `product_id` (string): The unique identifier for the product.
 - `product_name` (string): The name of the product.
 - `manufacturer` (string): The company that manufactures the product.
 - `price` (float): The retail price of the product.

- Implement an `__init__` method that initializes a `Product` with the attributes listed above.
 - Include the following methods in the `Product` class:
 - `display_info()`: Prints the product's details, including its ID, name, manufacturer, and price.
 - `apply_discount(discount_rate)`: Applies a percentage discount to the price and updates the price attribute.
2. **Laptop Class (Subclass):**
- Create a subclass named `Laptop` that inherits from `Product`. Add the following additional attributes:
 - `cpu` (string): The processor model in the laptop.
 - `ram` (integer): The amount of RAM in gigabytes.
 - `storage` (integer): The storage capacity in gigabytes.
 - `screen_size` (float): The size of the laptop's screen in inches.
 - Override the `__init__` method to include initialization of the new attributes while ensuring the inherited attributes are also initialized.
 - Override the `display_info()` method to include the laptop's specifications.
3. **Smartphone Class (Subclass):**
- Create a subclass named `Smartphone` that inherits from `Product`. Add the following additional attributes:
 - `camera_resolution` (float): The camera's resolution in megapixels.
 - `battery_life` (float): The estimated battery life in hours.
 - `water_resistant` (boolean): Indicates whether the smartphone is water-resistant.
 - Override the `__init__` method to include initialization of the new attributes while ensuring the inherited attributes are also initialized.
 - Override the `display_info()` method to include camera resolution, battery life, and water resistance.
4. **Testing Your Code:**
- Create instances of `Laptop` and `Smartphone` with relevant attributes.
 - Demonstrate the use of each method, including inherited and overridden methods, by interacting with both `Laptop` and `Smartphone` instances.

```
[134]: class Product:
    def __init__(self, product_id, product_name, manufacturer, price):
        # Basic stuff every product has.
        self.product_id = product_id
        self.product_name = product_name
        self.manufacturer = manufacturer
        self.price = price
    def display_info(self):
        print(f"Product ID: {self.product_id}")
        print(f"Product Name: {self.product_name}")
        print(f"Manufacturer: {self.manufacturer}")
        print(f"Price: ${self.price:.2f}")

    def apply_discount(self, discount_rate):
        # Taking off some percent.
```

```

        self.price = self.price * (1 - discount_rate / 100)

class Laptop(Product):
    def __init__(self, product_id, product_name, manufacturer, price, cpu, ram,
↪storage, screen_size):
        # Get the main product stuff from parent class.
        super().__init__(product_id, product_name, manufacturer, price)

        # Laptop specific attributes.
        self.cpu = cpu
        self.ram = ram
        self.storage = storage
        self.screen_size = screen_size

    def display_info(self):
        # Do the regular product info first.
        super().display_info()

        # Then the extra laptop things.
        print(f"CPU: {self.cpu}")
        print(f"RAM: {self.ram} GB")
        print(f"Storage: {self.storage} GB")
        print(f"Screen Size: {self.screen_size} inches")

class Smartphone(Product):
    def __init__(self, product_id, product_name, manufacturer, price,
↪camera_resolution, battery_life, water_resistant):
        # Same here.
        super().__init__(product_id, product_name, manufacturer, price)
        self.camera_resolution = camera_resolution
        self.battery_life = battery_life
        self.water_resistant = water_resistant

    def display_info(self):
        super().display_info()
        print(f"Camera Resolution: {self.camera_resolution} MP")
        print(f"Battery Life: {self.battery_life} hours")
        print(f"Water Resistant: {self.water_resistant}")

#
laptop1 = Laptop("L101", "MacBook Pro", "Apple", 1999.99, "M3 Pro", 18, 512, 14.
↪2)
phone1 = Smartphone("A1919", "iPhone 150", "Apple", 9999.99, 50.0, 24.0, True)

```

```

print("Laptop Info:")
laptop1.display_info()

print("\nApplying 60% discount to laptop...")
# Applying student discount
laptop1.apply_discount(60)
laptop1.display_info()

print("\nSmartphone Info:")
phone1.display_info()

```

Laptop Info:
 Product ID: L101
 Product Name: MacBook Pro
 Manufacturer: Apple
 Price: \$1999.99
 CPU: M3 Pro
 RAM: 18 GB
 Storage: 512 GB
 Screen Size: 14.2 inches

Applying 60% discount to laptop...
 Product ID: L101
 Product Name: MacBook Pro
 Manufacturer: Apple
 Price: \$800.00
 CPU: M3 Pro
 RAM: 18 GB
 Storage: 512 GB
 Screen Size: 14.2 inches

Smartphone Info:
 Product ID: A1919
 Product Name: iPhone 150
 Manufacturer: Apple
 Price: \$9999.99
 Camera Resolution: 50.0 MP
 Battery Life: 24.0 hours
 Water Resistant: True

3.2 Question 2: Safely Writing to a File (10 Points)

Task Develop a Python program that prompts users for their feedback and safely appends this feedback to a file named `feedback.txt`. The program should handle errors that may occur during

the file writing process without losing data.

Requirements

1. Collecting Feedback:

- Use the `input()` function to ask users to enter their feedback.

2. Writing to File:

- Append the feedback to `feedback.txt`. Each piece of feedback should be written on a new line.

3. Error Handling:

- Handle I/O errors that might occur while writing to the file by catching `IOError` and printing “Error: Failed to write to the file.”
- Handle any other unexpected errors with a generic error message.

```
[135]: try:
    # Ask the user for feedback.
    feedback = input("Enter your feedback: ")

    # Open the file in append mode so it adds onto the end.
    with open("feedback.txt", "a") as file:
        # Write the feedback and move to a new line.
        file.write(feedback + "\n")

    # Let them know it worked.
    print("Feedback saved successfully.")

except IOError:
    # File stuff broke somehow.
    print("Error: Failed to write to the file.")

except Exception as e:
    # Catch anything else weird.
    print("An unexpected error occurred.")
```

Feedback saved successfully.

3.3 Question 3: Validating User Input (10 Points)

Task Write a Python function named `validate_user_age` that prompts the user to enter their age and validates that it is a positive integer. The function should handle errors appropriately and provide clear error messages.

Requirements

1. User Input:

- Prompt the user to enter their age using the `input()` function.
2. **Validation:**
 - Check that the entered age is a positive integer.
 - Print “Age validated successfully.” if the input is correct.
 3. **Error Handling:**
 - If the input is not an integer, catch a `ValueError` and print “Error: Age must be a number.”
 - If the input is a negative integer, print “Error: Age cannot be negative.”
 - Handle any other unexpected error by printing “An unspecified error occurred.” followed by the error message.

```
[136]: def validate_user_age():
    try:
        # Ask the user for their age.
        age = int(input("Enter your age: "))

        # Check if they typed some cursed negative age.
        if age < 0:
            print("Error! Age cannot be negative.")
        else:
            # If it gets here, then it passed.
            print("Age validated successfully.")

    except ValueError:
        # This happens if they type words or weird stuff.
        print("Error! Age must be a number.")

    except Exception as e:
        # Catch anything else random.
        print("An unspecified error occurred.", e)

validate_user_age()
```

Age validated successfully.

3.4 Question 4: Summing Numbers from a File (10 Points)

Task Create a Python program named `sum_numbers.py` that reads numbers from a file named `numbers.txt`. Each line of the file will contain a single number. The program should calculate the sum of these numbers and print the total sum. Your program should also handle any file-related and input-related errors gracefully.

Requirements

1. **Reading Numbers:**
 - Open and read from a file called `numbers.txt`.
 - Assume each line in the file contains one number which could be an integer or a float.
2. **Calculating Sum:**
 - Calculate the sum of all the numbers read from the file.
3. **Error Handling:**
 - If `numbers.txt` does not exist, print: “Error: The file does not exist.”
 - If a line contains invalid data that cannot be converted to a number, print a warning: “Warning: Invalid number encountered and will be skipped.” and continue processing the rest of the file.
4. **Output:**
 - Print the total sum of all valid numbers at the end of the script execution.

```
[137]: try:
        # Start the total at 0.
        total = 0

        # Open the file and read through it.
        with open("numbers.txt", "r") as file:
            for line in file:
                # Clean it up a little.
                line = line.strip()

                try:
                    # Try turning the line into a number.
                    num = float(line)
                    total = total + num
                except ValueError:
                    # Bad line, skip it.
                    print("Warning: Invalid number encountered and will be skipped.
↵")

            # Print the final total.
            print("Total sum:", total)

        except FileNotFoundError:
            # File is gone.
            print("Error! The file does not exist.")
```

```
Warning: Invalid number encountered and will be skipped.
Total sum: 615.0
```

3.5 Question 5: Filtering Specific Lines from a File (10 Points)

Task Create a Python script named `filter_errors.py` that reads a file named `logs.txt` containing application logs. The script should print only the lines that contain the word “ERROR”. Additionally, the script must handle the case where the file does not exist.

Requirements

1. **Reading Logs:**
 - Open and read from a file called `logs.txt`.
2. **Filtering Lines:**
 - For each line in the file, check if the word “ERROR” is present.
 - If “ERROR” is found, print the line. Ensure that any trailing newline characters are removed before printing (use `.strip()` method).
3. **Error Handling:**
 - If `logs.txt` does not exist, print: “Error: The file does not exist.”

```
[138]: try:
        # Open the log file.
        with open("logs.txt", "r") as file:
            for line in file:
                # Check if ERROR shows up in the line.
                if "ERROR" in line:
                    # Strip the extra newline junk.
                    print(line.strip())

    except FileNotFoundError:
        # File is not there.
        print("Error! The file does not exist.")
```

ERROR: Failed to load resource.

ERROR: Database connection failed.

```
[ ]:
```